

Assignment 1: Observability — Report

Enterprise Software Development, Fall 2026 Repository: <https://github.com/romaisashazad/ESD-Assignment-01>

All times in screenshots are local time (UTC+5) unless they end in `Z`. Kibana and Grafana display local time, while the application writes UTC timestamps, so `17:56:52Z` in a log appears as `22:56:52` in Kibana.

Part A: Project

Problem

People often need to turn a piece of text or a URL into a QR code, for example for a menu, an event page or a flyer. Most free QR websites are full of ads, may track what users encode, and give no way to tell whether the service is fast or working.

Intended users

Small business owners and event organisers who need QR codes quickly, and developers who want a simple API to add QR generation to their own apps.

Solution

A small containerised web API written in Python with Flask. A client sends a JSON body containing `text`, and the service returns a PNG image of the QR code. The service is instrumented with Prometheus metrics and structured JSON logs, and runs alongside a full monitoring stack.

Endpoint	Purpose
<code>POST /generate</code>	Body <code>{"text": "..."} </code> returns a PNG QR code (<code>200</code>). A missing or empty <code>text</code> returns <code>400</code> . An unexpected failure returns <code>500</code> .
<code>GET /health</code>	Liveness check, <code>{"status": "ok"}</code>
<code>GET /metrics</code>	Metrics in Prometheus text format
<code>GET/POST /admin/fault</code>	Test only (Part E.1): switches the slow-request fault on or off
<code>POST /demo</code>	Test only (Part E.2): the cardinality demo counter

Every response includes an `X-Request-ID` header, so a user can report a problem and the matching log can be found.

What works and how to try it

The whole system is defined in `docker-compose.yml` and starts with `docker compose up -d --build`. This starts 7 containers: the app (`qr-service`), `prometheus`, `grafana`, `node-exporter`, `elasticsearch`, `kibana` and `filebeat`. All 7 run successfully (Figure A1).

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED   STATUS    PORTS
elasticsearch        docker.elastic.co/elasticsearch:8.15.0 "/bin/tini -- /usr/l..."          elasticsearch 18 hours ago Up 18 hours 0.0.0.0
:9200->9200/tcp, [::]:9200->9200/tcp
filebeat             docker.elastic.co/beats/filebeat:8.15.0 "/usr/bin/tini -- /u..."          filebeat      7 hours ago Up 7 hours 0.0.0.0
grafana              grafana/grafana:latest              "/run.sh"                                grafana        5 hours ago Up 5 hours 0.0.0.0
:3001->3000/tcp, [::]:3001->3000/tcp
kibana               docker.elastic.co/kibana/kibana:8.15.0 "/bin/tini -- /usr/l..."          kibana        18 hours ago Up 18 hours 0.0.0.0
:5601->5601/tcp, [::]:5601->5601/tcp
node-exporter        prom/node-exporter:latest            "/bin/node_exporter"                  node-exporter  19 hours ago Up 19 hours 0.0.0.0
:9100->9100/tcp, [::]:9100->9100/tcp
prometheus           prom/prometheus:latest               "/bin/prometheus --c..."            prometheus    5 hours ago Up 5 hours 0.0.0.0
:9090->9090/tcp, [::]:9090->9090/tcp
qr-service           esd-assignment-01-qr-service         "python app.py"                        qr-service    4 hours ago Up 4 hours 0.0.0.0
:5000->5000/tcp, [::]:5000->5000/tcp
PS C:\Users\IBM\Desktop\ESD-Assignment-01> |
```

Figure A1: `docker compose ps`, all 7 containers up.

Sending `{"text": "hello from postman"}` to POST `http://localhost:5000/generate` returns 200 OK with the QR code image (Figure A2). A 30-iteration Postman run completed with no errors and an average of 18 ms (Figure A3). Sending `{}` returns 400 `{"error": "Missing 'text' field"}`.

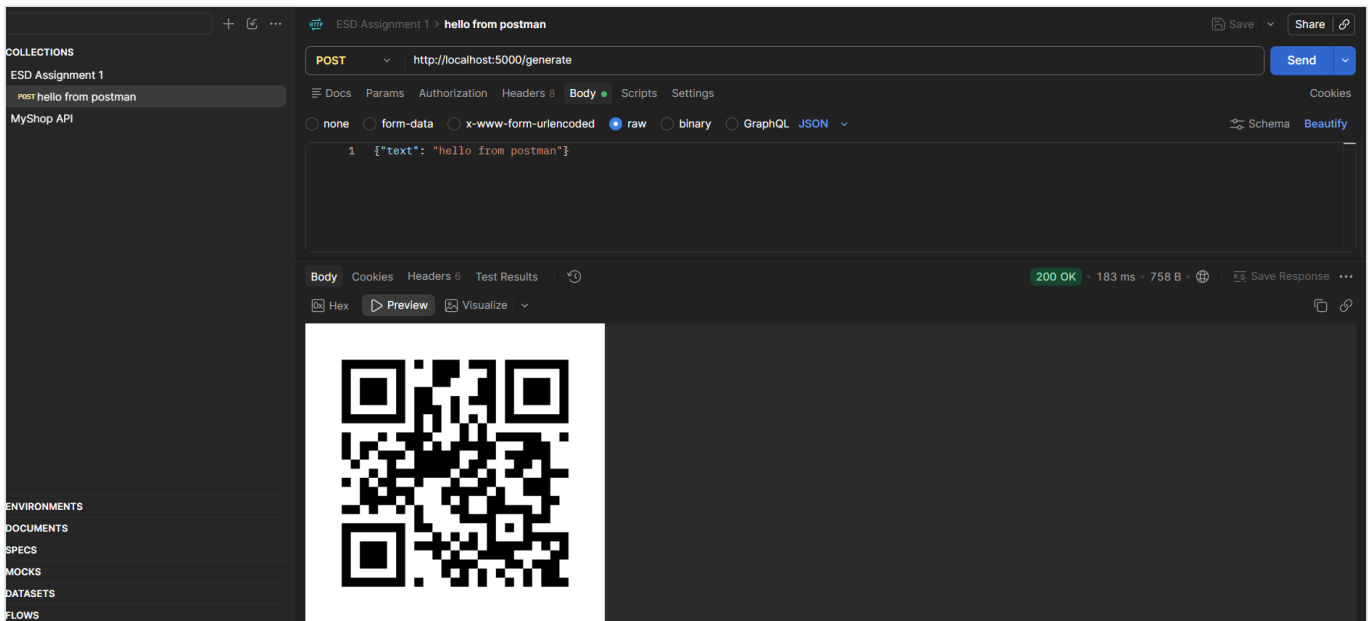


Figure A2: Postman request and the returned QR code.

ESD Assignment 1 - Run results

Rerun

New Run

Schedule

Postman CLI

Configure CI/CD

Share

...

Ran today at 02:16:24 AM

View all runs

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	30	5s 965ms	0	0	18 ms

All 0

Passed 0

Failed 0

Skipped 0

Errors 0

Console log

List

Grid

Iteration 27

POST

hello from postman

http://localhost:5000/generate

200

• 11 ms • 756 B

No tests found

Iteration 28

POST

hello from postman

http://localhost:5000/generate

200

• 10 ms • 756 B

No tests found

Iteration 29

POST

hello from postman

http://localhost:5000/generate

200

• 14 ms • 756 B

No tests found

Iteration 30

POST

hello from postman

http://localhost:5000/generate

200

• 64 ms • 756 B

No tests found

Figure A3: Postman Runner, 30 successful iterations.

Full start, use, test and clean-up instructions are in `README.md`.

Part B: Metrics

Setup

Prometheus scrapes two targets every 5 seconds: `qr-service:5000/metrics` (application metrics) and `node-exporter:9100/metrics` (host metrics). Both are UP (Figure B1). Prometheus stores its data in the named volume `prometheus-data` and keeps it for 7 days (`--storage.tsdb.retention.time=7d`).

Grafana is provisioned from files in the repo. `telemetry/grafana/provisioning/` defines the Prometheus datasource, and `telemetry/grafana/dashboards/qr-service.json` is the dashboard. A fresh `docker compose up` therefore always shows the full dashboard with no manual setup (Figure B8). The dashboard has 10 panels: 6 application and business panels and 4 host panels.

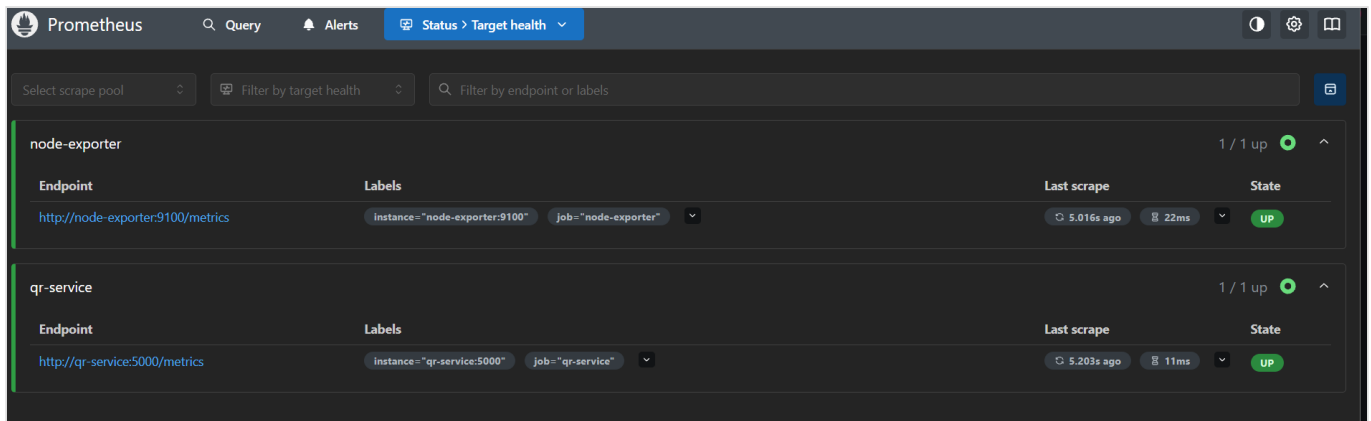


Figure B1: Prometheus targets, both UP.

Metrics table

All application metrics are defined and recorded in `services/qr-service/app.py`. All four metric types are used.

Metric	Type	Category	Purpose	Unit	Labels	Where and how it is recorded
<code>qr_generated_total</code>	Counter	Business	QR codes successfully created (like "orders placed")	count	<code>kind = url / text</code>	<code>generate(): .labels(kind=kind).inc()</code> after the PNG is saved
<code>qr_generation_errors_total</code>	Counter	Application	Failed requests, by cause	count	<code>reason = missing_text / internal_error</code>	<code>generate():</code> in the 400 branch and in the <code>except</code> branch
<code>qr_in_progress</code>	Gauge	Application	Requests being processed right now	count	none	<code>generate(): .inc()</code> before generating, <code>.dec()</code> in finally

Metric	Type	Category	Purpose	Unit	Labels	Where and how it is recorded
<code>qr_generation_duration_seconds</code>	Histogram	Application	Distribution of generation time; used for p95	seconds	le buckets 0.01 to 5 s	<code>generate(): .observe(duration)</code> after success
<code>qr_generation_duration_summary_seconds</code>	Summary	Application	Count and sum of generation time; used for the average	seconds	none	Same place as the histogram
<code>qr_fault_injection_active</code>	Gauge	Application (test)	1 while the Part E fault is on	0 / 1	none	<code>admin_fault(): .set(1)</code> or <code>.set(0)</code>
<code>demo_requests_total</code>	Counter	Test only (E.2)	Cardinality demo	count	<code>request_id</code> (only when the demo is on)	<code>demo()</code>
<code>node_cpu_seconds_total</code>	Counter	Host	CPU time per core and mode	seconds	<code>cpu</code> , <code>mode</code>	Node Exporter
<code>node_memory_MemAvailable_bytes</code> , <code>node_memory_MemTotal_bytes</code>	Gauge	Host	Free and total memory	bytes	none	Node Exporter
<code>node_filesystem_avail_bytes</code> , <code>node_filesystem_size_bytes</code>	Gauge	Host	Free and total disk	bytes	<code>mountpoint</code> , <code>device</code> , <code>fstype</code>	Node Exporter
<code>node_network_receive_bytes_total</code>	Counter	Host	Bytes received per interface	bytes	<code>device</code>	Node Exporter

Implementation notes:

- `duration` is measured from just before `qrcode.make()` until the PNG is written to memory. It does not include network time or Flask's own overhead, which matters in Part E.
- The `kind` and `reason` labels have only two values each. Every label combination is created at 0 when the app starts, so panels show 0 instead of "No data" before the first event.
- The Python client also exports a `_created` timestamp series next to each counter, which becomes important in Part E.2.
- Only the length of the user's text is recorded, never the text itself.

The raw `/metrics` output with the labelled series is shown in Figures B5a and B5b.

```

← → ↺ 📄 localhost:5000/metrics

# HELP python_gc_objects_collected_total Objects collected during gc
# TYPE python_gc_objects_collected_total counter
python_gc_objects_collected_total{generation="0"} 442.0
python_gc_objects_collected_total{generation="1"} 83.0
python_gc_objects_collected_total{generation="2"} 0.0
# HELP python_gc_objects_uncollectable_total Uncollectable objects found during GC
# TYPE python_gc_objects_uncollectable_total counter
python_gc_objects_uncollectable_total{generation="0"} 0.0
python_gc_objects_uncollectable_total{generation="1"} 0.0
python_gc_objects_uncollectable_total{generation="2"} 0.0
# HELP python_gc_collections_total Number of times this generation was collected
# TYPE python_gc_collections_total counter
python_gc_collections_total{generation="0"} 76.0
python_gc_collections_total{generation="1"} 6.0
python_gc_collections_total{generation="2"} 0.0
# HELP python_info Python platform information
# TYPE python_info gauge
python_info{implementation="CPython",major="3",minor="12",patchlevel="14",version="3.12.14"} 1.0
# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 2.02764288e+08
# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 3.8928384e+07
# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.79016428937e+09
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 4.17
# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 6.0
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1.048576e+06
# HELP qr_generated_total Total number of QR codes generated
# TYPE qr_generated_total counter
qr_generated_total{kind="url"} 1.0
qr_generated_total{kind="text"} 1.0
# HELP qr_generated_created Total number of QR codes generated
# TYPE qr_generated_created gauge
qr_generated_created{kind="url"} 1.7901642927259119e+09
qr_generated_created{kind="text"} 1.790164292725923e+09
# HELP qr_generation_errors_total Total number of failed QR generation requests
# TYPE qr_generation_errors_total counter
qr_generation_errors_total{reason="missing_text"} 1.0

```

```

# HELP qr_generation_errors_total Total number of failed QR generation requests
# TYPE qr_generation_errors_total counter
qr_generation_errors_total{reason="missing_text"} 1.0
qr_generation_errors_total{reason="internal_error"} 0.0
# HELP qr_generation_errors_created Total number of failed QR generation requests
# TYPE qr_generation_errors_created gauge
qr_generation_errors_created{reason="missing_text"} 1.7901642927259312e+09
qr_generation_errors_created{reason="internal_error"} 1.7901642927259376e+09
# HELP qr_in_progress Number of QR generation requests currently in progress
# TYPE qr_in_progress gauge
qr_in_progress 0.0
# HELP qr_generation_duration_seconds Time taken to generate a QR code, in seconds
# TYPE qr_generation_duration_seconds histogram
qr_generation_duration_seconds_bucket{le="0.01"} 0.0
qr_generation_duration_seconds_bucket{le="0.025"} 1.0
qr_generation_duration_seconds_bucket{le="0.05"} 1.0
qr_generation_duration_seconds_bucket{le="0.1"} 1.0
qr_generation_duration_seconds_bucket{le="0.25"} 1.0
qr_generation_duration_seconds_bucket{le="0.5"} 1.0
qr_generation_duration_seconds_bucket{le="1.0"} 1.0
qr_generation_duration_seconds_bucket{le="2.5"} 2.0
qr_generation_duration_seconds_bucket{le="5.0"} 2.0
qr_generation_duration_seconds_bucket{le="+Inf"} 2.0
qr_generation_duration_seconds_count 2.0
qr_generation_duration_seconds_sum 1.194051742553711
# HELP qr_generation_duration_seconds_created Time taken to generate a QR code, in seconds
# TYPE qr_generation_duration_seconds_created gauge
qr_generation_duration_seconds_created 1.790164292725841e+09
# HELP qr_generation_duration_summary_seconds Summary of QR code generation duration, in seconds
# TYPE qr_generation_duration_summary_seconds summary
qr_generation_duration_summary_seconds_count 2.0
qr_generation_duration_summary_seconds_sum 1.194051742553711
# HELP qr_generation_duration_summary_seconds_created Summary of QR code generation duration, in seconds
# TYPE qr_generation_duration_summary_seconds_created gauge
qr_generation_duration_summary_seconds_created 1.7901642927258897e+09

```

Figure B5: Raw `/metrics` output, showing the `kind` and `reason` series, histogram buckets and the summary.

```
# TYPE qr_generation_errors_total counter
# HELP qr_in_progress Number of QR generation requests currently in progress
# TYPE qr_in_progress gauge
qr_in_progress 0.0
# HELP qr_generation_duration_seconds Time taken to generate a QR code, in seconds
# TYPE qr_generation_duration_seconds histogram
qr_generation_duration_seconds_bucket{le="0.01"} 31.0
qr_generation_duration_seconds_bucket{le="0.025"} 31.0
qr_generation_duration_seconds_bucket{le="0.05"} 32.0
qr_generation_duration_seconds_bucket{le="0.1"} 34.0
qr_generation_duration_seconds_bucket{le="0.25"} 34.0
qr_generation_duration_seconds_bucket{le="0.5"} 34.0
qr_generation_duration_seconds_bucket{le="1.0"} 34.0
qr_generation_duration_seconds_bucket{le="2.5"} 34.0
qr_generation_duration_seconds_bucket{le="5.0"} 34.0
qr_generation_duration_seconds_bucket{le="+Inf"} 34.0
qr_generation_duration_seconds_count 34.0
qr_generation_duration_seconds_sum 0.3110206127166748
# HELP qr_generation_duration_seconds_created Time taken to generate a QR code, in seconds
# TYPE qr_generation_duration_seconds_created gauge
qr_generation_duration_seconds_created 1.7901056432561896e+09
# HELP qr_generation_duration_summary_seconds Summary of QR code generation duration, in seconds
# TYPE qr_generation_duration_summary_seconds summary
qr_generation_duration_summary_seconds_count 34.0
qr_generation_duration_summary_seconds_sum 0.3110206127166748
# HELP qr_generation_duration_summary_seconds_created Summary of QR code generation duration, in seconds
# TYPE qr_generation_duration_summary_seconds_created gauge
qr_generation_duration_summary_seconds_created 1.7901056432563164e+09
```

Figure B5c: Histogram buckets from an earlier run of 34 requests.

Grafana panels and queries

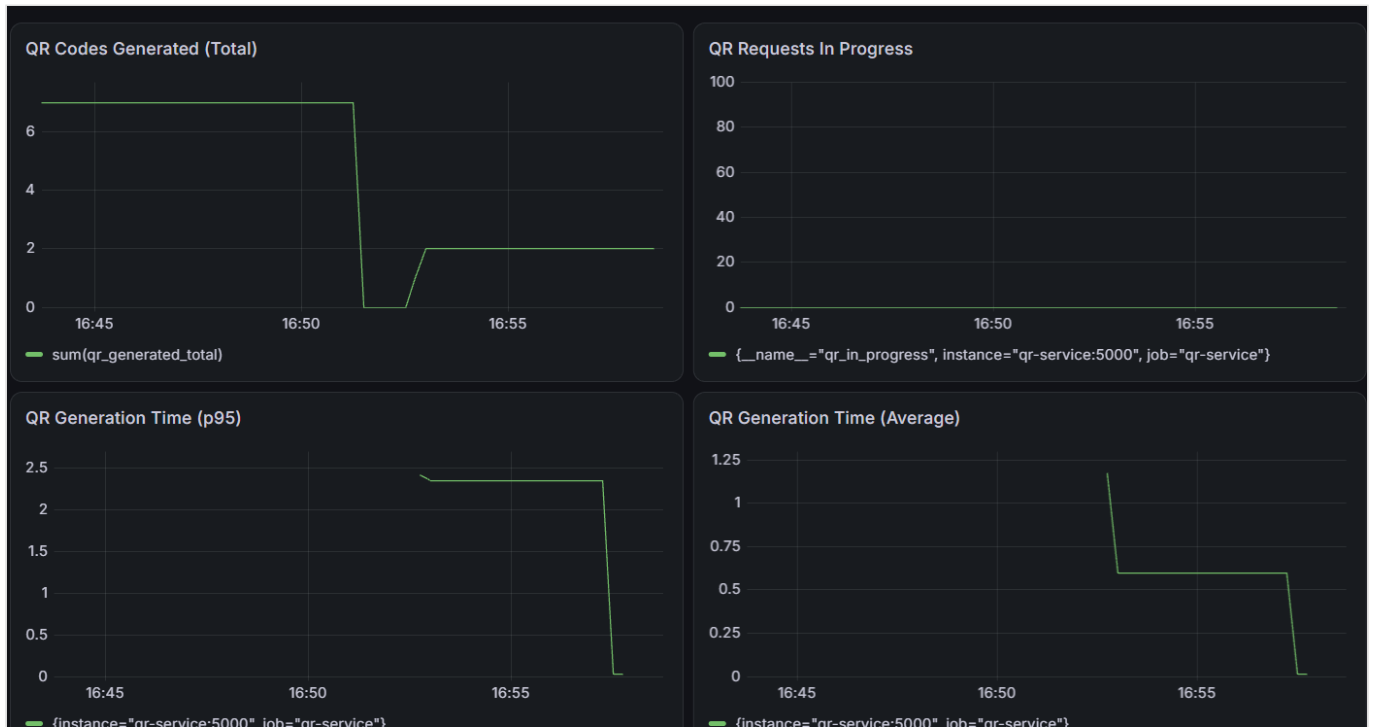


Figure B2: Application panels (16:43–16:58).



Figure B3: Business and error panels.

Panel 1: QR Codes Generated (Total)

```
sum(qr_generated_total)
```

This shows the raw running total, summed over both `kind` values. It rises in steps during bursts of traffic and stays flat in between. The sharp drop to 0 at 16:51 in Figure B2 is a counter reset: the app was rebuilt at that moment. Counters live in the app's memory, so every restart starts them at 0 again. Figure B7 shows the same effect over 6 hours of development, with drops at about 13:25 and 14:30 at each rebuild. This is why rate-based queries (Panels 3–6) use `rate()`, which detects resets and ignores them, while a raw counter panel can be misleading.

Panel 2: QR Requests In Progress

```
qr_in_progress
```

This stays at 0 almost all the time. A request takes about 10 ms, and Prometheus only samples the gauge once every 5 seconds, so it almost never catches a request in progress. This is correct behaviour for a fast service. The gauge would matter if requests became slow or piled up.

Panel 3: QR Generation Time (p95)

```
histogram_quantile(0.95, rate(qr_generation_duration_seconds_bucket[5m]))
```

Time window: at every point, `rate(...[5m])` looks at how much each bucket grew during the previous 5 minutes. So the value at 21:55 means "95% of QR codes generated between 21:50 and 21:55 took this long or less." When there is no traffic in the window, there is no value, which is why the line has gaps.

The p95 is an estimate. Prometheus only knows how many requests fell into each bucket, not their exact durations. It finds the bucket that contains the 95th percentile and assumes the values are spread evenly inside it. For example, in Figure B5c, 31 of 34 requests were under 0.01 s. The 95th-percentile request (the 32.3rd of 34) falls in the 0.05–0.1 s bucket, so Prometheus reports a value inside that bucket. Part E shows that this estimate can be well above the real value.

Cold starts: the first requests after a restart are much slower, 0.6–2.4 s, while Python loads the image libraries. That is why p95 briefly jumps after every rebuild (for example, 2.4 s at 16:53 in Figure B2). With only a few requests in the window, one slow request dominates the p95.

Panel 4: QR Generation Time (Average)

```
rate(qr_generation_duration_summary_seconds_sum[5m]) /  
rate(qr_generation_duration_summary_seconds_count[5m])
```

This divides the total time spent by the number of requests over the same 5-minute window, giving the average time per QR code. As the assignment notes, Python summaries do not compute percentiles, so the summary is used only for the average, and p95 comes from the histogram in Panel 3.

Panel 5: QR Codes per Minute by Type *(a metric I explored myself)*

```
sum by (kind) (rate(qr_generated_total[5m])) * 60
```

This splits production into URL and plain-text QR codes, which answers a real business question: what users actually encode. It also handles counter resets correctly, because it uses `rate()`. During the Part E load test, the two lines sit on top of each other at 50 per minute each, because the load script alternates URL and text requests.

Panel 6: Error Rate (%) *(a metric I explored myself)*

```
100 * sum(rate(qr_generation_errors_total[5m])) / (sum(rate(qr_generated_total[5m])) +  
sum(rate(qr_generation_errors_total[5m])))
```

This is the share of all requests that failed in the last 5 minutes. In Figure B3 it steps to 33% and then 100%, because only a few manual test requests were sent, some of them deliberately invalid (`{}`). When there is no traffic at all, the result is $0 \div 0$, so the panel shows a gap rather than a value.

Panel 7: CPU Usage (%)

```
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
```

This takes the fraction of time each core spent idle, averages it across the 4 cores, and subtracts from 100. The spikes to 100% in Figure B7 match container rebuilds and restarts, when images are built and Elasticsearch starts up. They are not caused by QR traffic: a 30-request Postman run adds less than a second of CPU work. During the steady load in Figure B4, CPU rises from about 10% to about 25%.

Panel 8: Memory Usage (%)

```
100 * (1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes))
```

Memory use is about 62–66%. Before Elasticsearch and Kibana were added, it was about 37–42%, so the logging stack costs about a quarter of the VM's 4.8 GB.

Panel 9: Disk Usage (%)

```
100 - ((node_filesystem_avail_bytes{mountpoint="/etc/hostname"} /  
node_filesystem_size_bytes{mountpoint="/etc/hostname"}) * 100)
```

Disk use is about 5.9% of the VM's disk (`/dev/sdg`, `ext4`). Node Exporter runs inside a container, so it cannot see the host's `/` directly. It only sees the files Docker bind-mounts into it (`/etc/hostname`, `/etc/hosts`, `/etc/resolv.conf`). All three sit on the same disk as Docker's data, so any of them gives the correct disk usage. `/etc/hostname` was chosen for that reason.

Panel 10: Network Traffic (bytes/sec)

```
rate(node_network_receive_bytes_total{device!="lo"}[5m])
```

This is a known limitation. The only interface is `eth0` of the node-exporter container itself, because the container has its own network namespace. The steady ~ 120 B/s is essentially Prometheus scraping Node Exporter every 5 seconds, and it does not show the app's traffic. Measuring real host traffic would need `network_mode: host`, which works differently on Docker Desktop for Windows, so I documented the limitation instead.

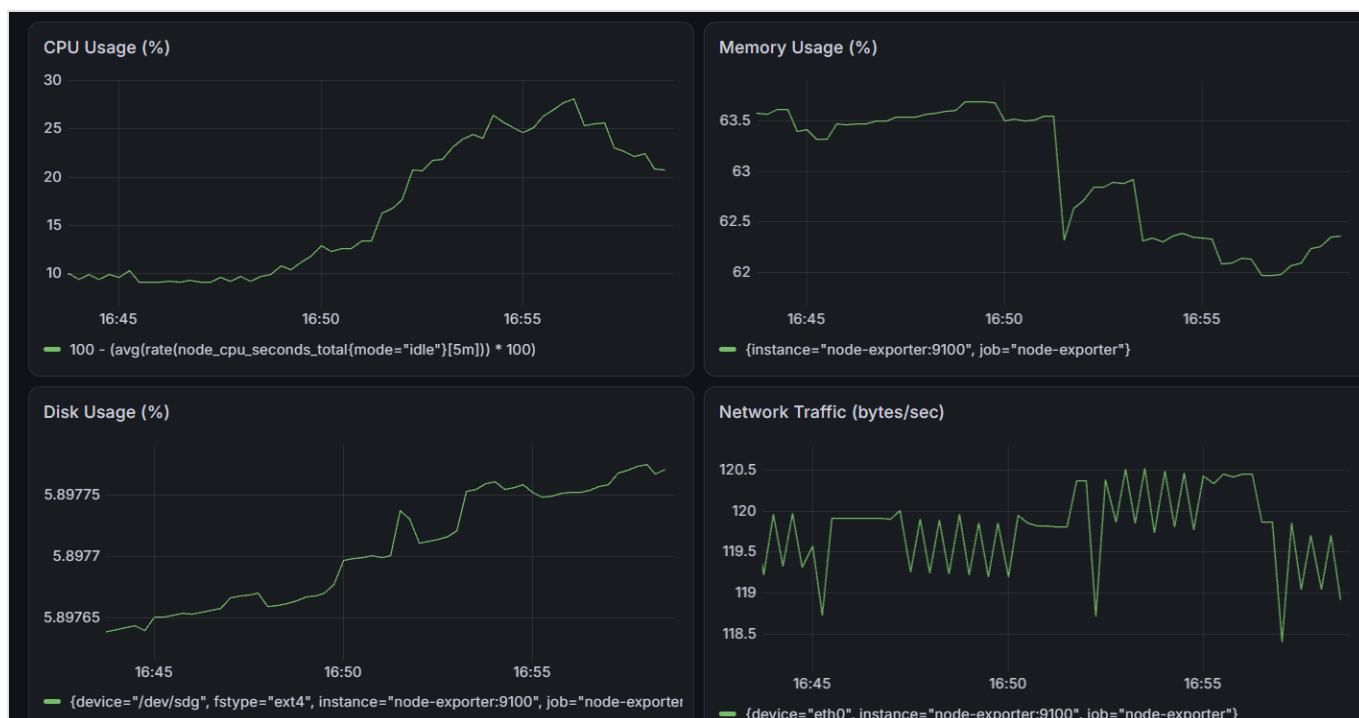


Figure B4: Host panels (16:43-16:58).



Figure B7: 6-hour view. The counter drops to 0 at each app rebuild, and CPU spikes match the rebuilds.

Machine being measured

Node Exporter reports what it can see from inside Docker (Figure B6):

- **Kernel:** `node_uname_info` shows `release="6.6.87.2-microsoft-standard-WSL2"`, `sysname="Linux"`, `machine="x86_64"`.
- **CPUs:** `count(node_cpu_seconds_total{mode="idle"}) = 4`.
- **Memory:** `node_memory_MemTotal_bytes / 1024^3 ≈ 4.8 GB`.

The machine measured is therefore **Docker Desktop's WSL2 Linux virtual machine** (4 vCPUs, about 4.8 GB of RAM), running on the Windows laptop **DESKTOP-2HIMGAT**. Node Exporter sees the VM's resources, not the whole Windows machine. Its `nodename` label (`325c239dc047`) is only its own container ID.

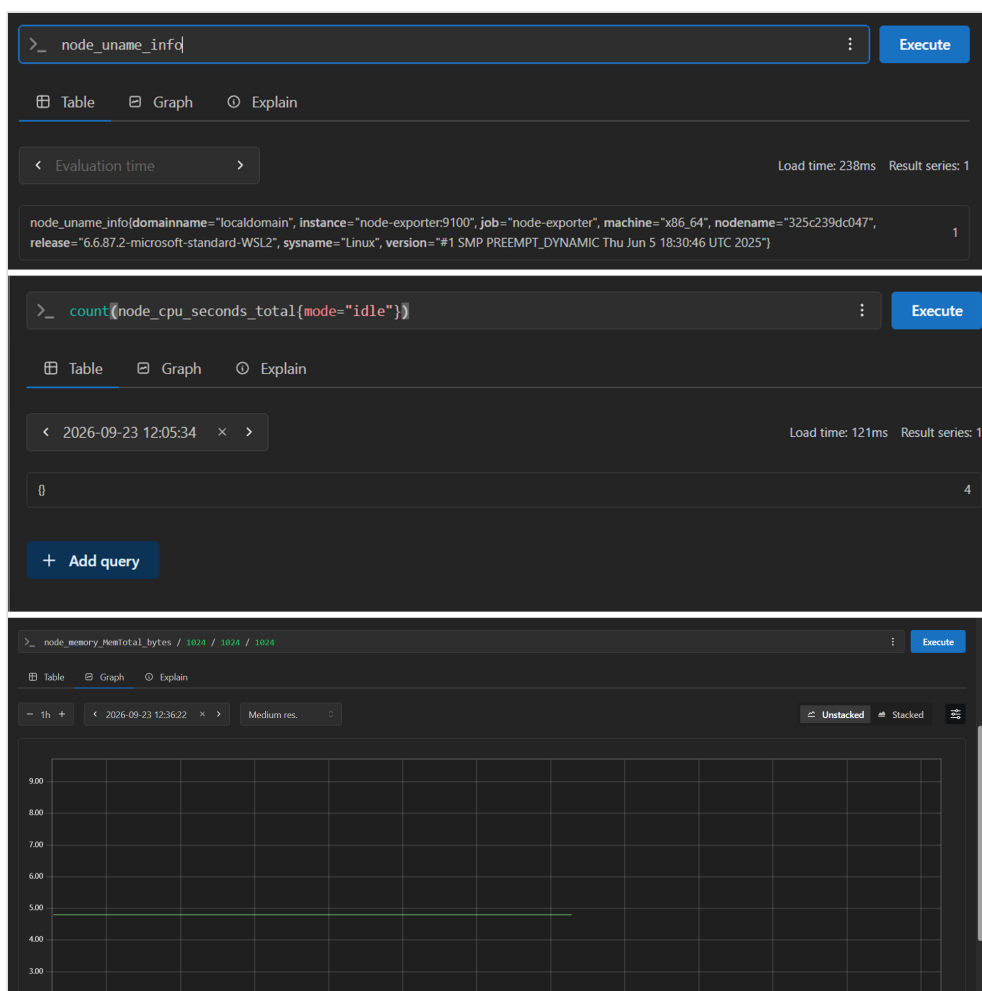


Figure B6: `node_uname_info`, CPU count and total memory.

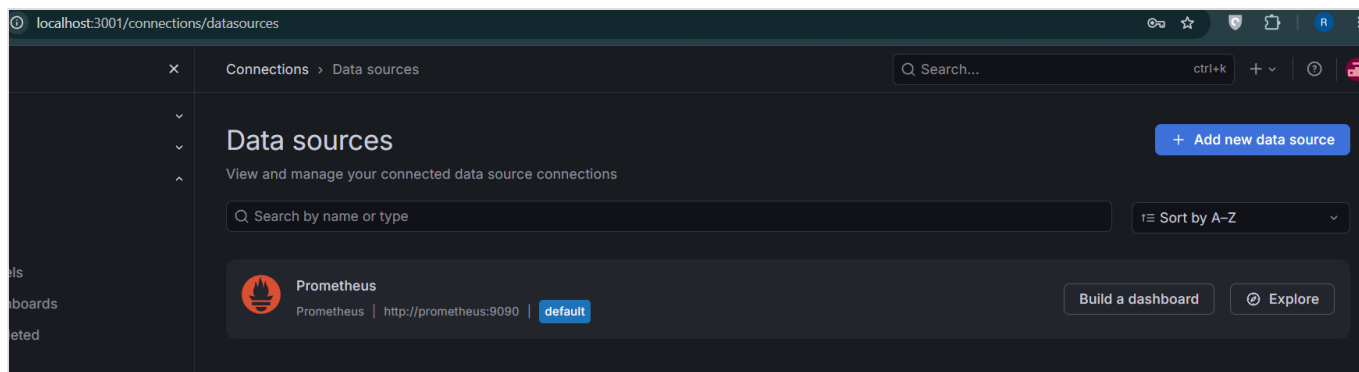


Figure B8: The Prometheus datasource, loaded from provisioning files.

Part C: Logs

Pipeline

```
app.py writes one JSON line to stdout
→ Docker's json-file driver saves it to /var/lib/docker/containers/<id>/<id>-json.log
→ Filebeat reads the file, unwraps and parses it
→ Elasticsearch stores it in the daily data stream qr-service-logs-YYYY.MM.DD
→ Kibana searches it (data view "QR Service Logs", pattern qr-service-logs-*)
```

Logstash is not used. Filebeat's built-in processors do all the parsing this project needs.

What is logged, why, and where in the code

All logs are written by `log_event()` in `app.py`, which builds a dictionary and prints it as one JSON line to stdout.

Event	Where	Level	Why
QR generated	<code>generate()</code> , success path	INFO	Record of every successful request, with its duration
Missing <code>text</code>	<code>generate()</code> , 400 branch	ERROR	Shows client mistakes
Unexpected exception	<code>generate()</code> , except branch	ERROR	Includes <code>error.message</code> for debugging
Fault switched on or off	<code>admin_fault()</code>	WARN / INFO	Records exactly when the Part E fault was active

Fields in every log: `@timestamp` (UTC), `service.name`, `log.level`, `message` and `request.id`. Request logs also include `http.method`, `http.path` and `http.response.status_code`. Successful requests add `duration.ms`, `qr.kind` and `text.length`, and delayed requests add `fault.delay_ms`.

Request IDs: `assign_request_id()` runs before every request. It uses the client's `X-Request-ID` header if one was sent, and otherwise generates a UUID. `return_request_id()` sends the ID back in the response header, so the ID a user sees matches the one in the log.

What is deliberately not logged:

- **The user's text.** It could contain private URLs or personal data, so only `text.length` and `qr.kind` are logged. No secrets exist in the app, and none are logged.

- **Werkzeug's access log.** Flask's development server printed a plain-text line for every request, including every Prometheus scrape of `/metrics` (12 per minute). Before I turned it off with `logging.getLogger("werkzeug").setLevel(logging.ERROR)`, **519 of 550 stored documents (94%) were these scrape lines**, with no parsed fields. They were noise that buried the real logs, so they were removed rather than parsed.

How Filebeat collects and parses logs

The configuration is in `telemetry/filebeat/filebeat.yml`:

1. **The container input** reads `/var/lib/docker/containers/**/*.log`. This directory is mounted read-only into Filebeat. Docker stores each line wrapped in its own JSON, as `{"log": "<our line>", "stream": "stdout", "time": "..."}` , and the container input unwraps it, putting our line into `message`.
2. **add_docker_metadata** asks the Docker socket which container the file belongs to and adds `container.name`, `container.id`, `container.image.name` and the container labels.
3. **decode_json_fields** (`target: ""`, `overwrite_keys: true`) parses `message` as JSON and moves our fields to the top level. `message` becomes our own text ("QR code generated successfully"), and `@timestamp` becomes our app's timestamp.
4. **drop_event** discards every event where `container.name` is not `qr-service`, so logs from Elasticsearch, Kibana and the others are not stored.

Filebeat also adds its own fields: `agent.*`, `host.name` (Filebeat's container ID), `input.type`, `log.file.path`, `log.offset` (the byte position in the file) and `ecs.version`.

Docker autodiscover was tried first but did not detect containers reliably on Docker Desktop/WSL2, so the simpler container input with a filter is used instead. The trade-off is that Filebeat reads every container's logs and then throws most of them away, which would be wasteful on a busy host.

One original log, its stored fields, and a working search

This traces one request (ID `5d0697ce-2075-45e3-8592-cf5fa7363df1`) from the client to Kibana.

The client receives the ID in the `X-Request-ID` header (Figure C1).

The original log, as the app wrote it (`docker logs qr-service | Select-String "5d0697ce-..."`, Figure C2):

```
{"@timestamp": "2026-09-23T15:52:06Z", "service.name": "qr-service", "log.level": "INFO",
"message": "QR code generated successfully", "http.method": "POST", "http.path": "/generate",
"http.response.status_code": 200, "duration.ms": 160.87, "request.id": "5d0697ce-2075-45e3-8592-cf5fa7363df1", "qr.kind": "url", "text.length": 19}
```

The stored fields in Elasticsearch (Kibana document Table view, Figures C4a and C4b) include all of the app's fields as separate searchable fields. `duration.ms`, `http.response.status_code` and `text.length` are stored as numbers, so range queries work. They sit alongside the fields Filebeat added: `container.name: qr-service`, `log.file.path: /var/lib/docker/containers/e7b9.../e7b9...-json.log`, `log.offset: 1,445`, `stream: stdout` and `input.type: container`.

The working search is `request.id:"5d0697ce-2075-45e3-8592-cf5fa7363df1"`, which returns exactly 1 document (Figure C3).

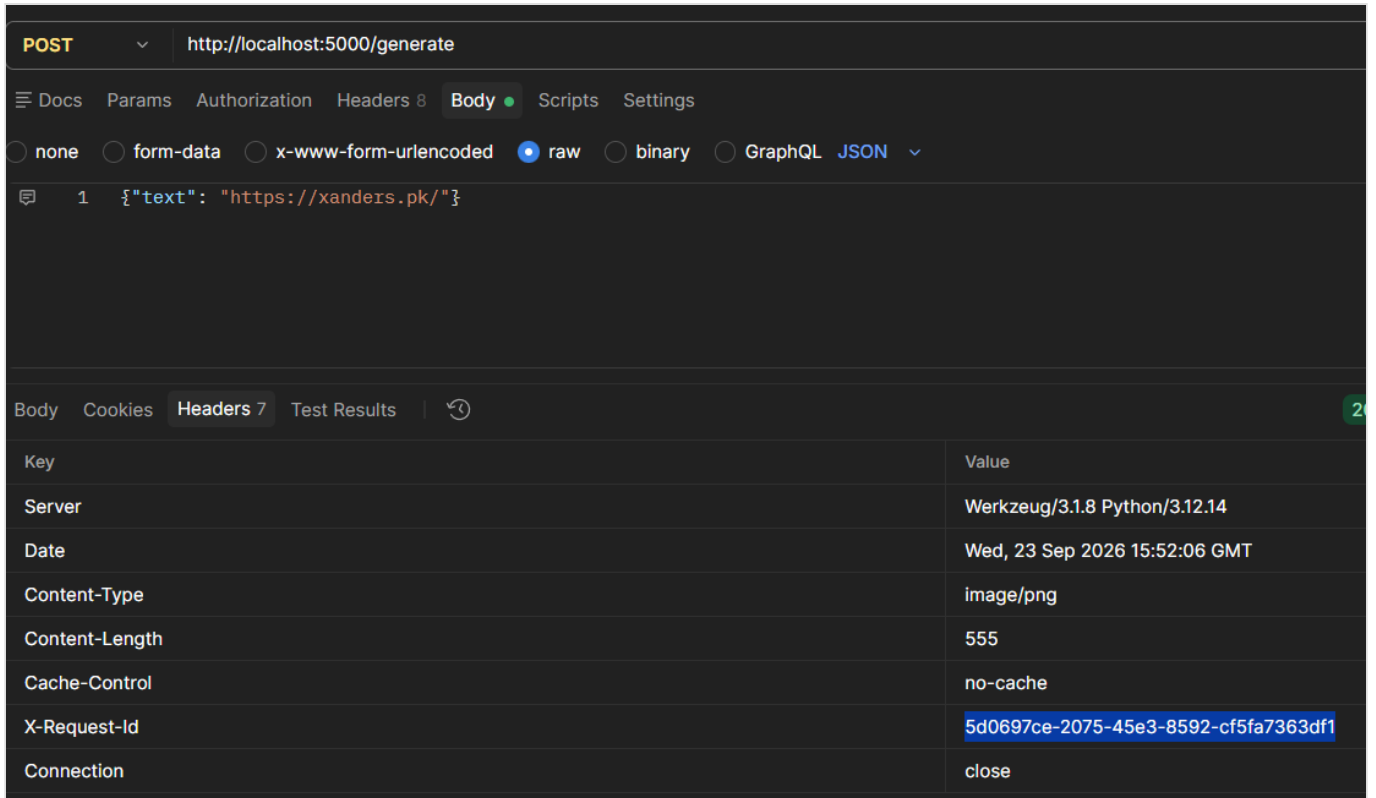


Figure C1: The request ID in Postman's response headers.

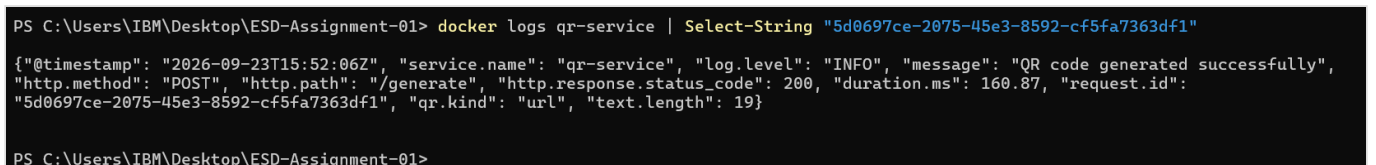


Figure C2: The original log line from `docker logs`.

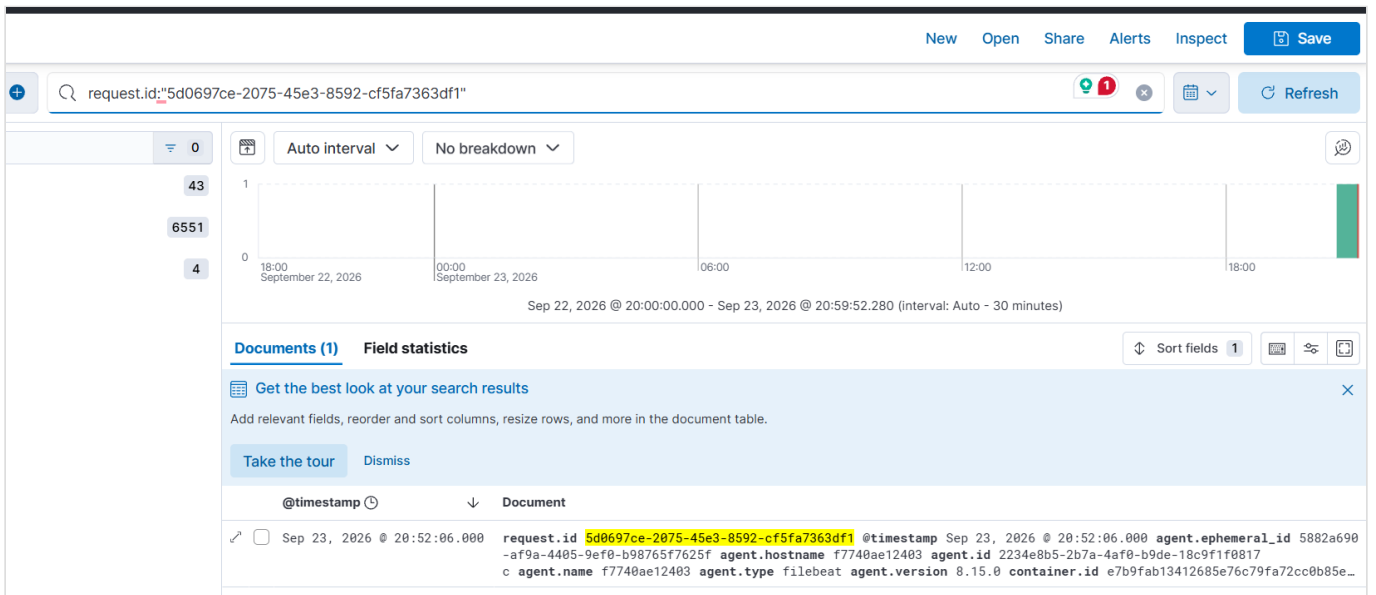


Figure C3: The Kibana search by request ID, 1 hit.

D Discover .ds-qr-service-logs-2026.09.23-2026.09.23-000001#Mer3zqABLb2BTuTFxUBu	
Table	JSON
Search field names	
Field	Value
k _id	Mer3zqABLb2BTuTFxUBu
k _ignored	-
k _index	.ds-qr-service-logs-2026.09.23-2026.09.23-000001
# _score	0
@timestamp	Sep 23, 2026 @ 20:52:06.000
k agent.ephemeral_id	5882a690-af9a-4405-9ef0-b98765f7625f
k agent.hostname	f7740ae12403
k agent.id	2234e8b5-2b7a-4af0-b9de-18c9f1f0817c
k agent.name	f7740ae12403
k agent.type	filebeat
k agent.version	8.15.0
k container.id	e7b9fab13412685e76c79fa72cc0b85e41da20dd2c5efdbe3a34c5b9da1ded21
k container.image.name	esd-assignment-01-qr-service
k container.labels.co_elastic_logs/enabled	true
k container.labels.com_docker_compose_config-hash	3ba6ec22b7d509a6b61373588b8ae3026a21928d09fc1dfeadd4bb0efd14fc31

Search field names	
Field	Value
k container.labels.desktop_docker_io/ports_scheme	v2
k container.labels.desktop_docker_io/ports/5000/tcp	:5000
k container.name	qr-service
# duration.ms	160.87
k ecs.version	8.0.0
k host.name	f7740ae12403
k http.method	POST
k http.path	/generate
# http.response.status_code	200
k input.type	container
k log.file.path	/var/lib/docker/containers/e7b9fab13412685e76c79fa72cc0b85e41da20dd2c5efdbe3a34c5b9da1ded21/e7b9fab13412685e76c79fa72cc0b85e41da20dd2c5efdbe3a34c5b9da1ded21-json.log
k log.level	INFO
# log.offset	1,445
t message	QR code generated successfully
k qr.kind	url
k request.id	5d0697ce-2075-45e3-8592-cf5fa7363df1
k service.name	qr-service
k stream	stdout
# text.length	19

Figure C4: Stored fields in Elasticsearch.

Where logs live, what survives restarts, and when old logs are deleted

Layer	Where	Survives container restart?	Survives docker compose down ?	When deleted
Docker log file	/var/lib/docker/containers/<id>/inside the Docker VM	Yes	No, the file belongs to the container	Rotated at 10 MB, keeping 3 files (about 30 MB), oldest deleted first. Also deleted whenever the container is removed or rebuilt.
Filebeat registry (read position)	Volume <code>filebeat-data</code>	Yes	Yes	Only with <code>down -v</code>
Elasticsearch documents	Volume <code>esdata</code>	Yes	Yes	Never automatically. ILM is disabled (<code>setup.ilm.enabled: false</code>). Deleted manually one day at a time, or all together with <code>down -v</code> .
Kibana data view and saved searches	Stored inside Elasticsearch (<code>esdata</code>)	Yes	Yes	With <code>down -v</code> (re-import <code>telemetry/kibana/saved-objects.ndjson</code>)

Docker's log file is short-term storage, and Elasticsearch is the long-term store. Because each day is its own data stream, a day of logs can be deleted in one call. I did this during development:

```
Invoke-RestMethod -Method Delete "http://localhost:9200/_data_stream/qr-service-logs-2026.09.22"
```

In production, an ILM or data-stream retention policy, for example "delete after 30 days", would automate this.

Two fixes made during this part:

- **Filebeat registry volume.** Filebeat records how far it has read each file in its registry. Without a volume, recreating the Filebeat container lost that position, and Filebeat re-sent every log from the beginning, creating duplicates. After adding `filebeat-data`, the test was: `count = 37`, then `docker compose up -d --force-recreate filebeat`, then `count = 37` again. New requests afterwards raised it to 39 and 40, one per request (Figure C6).
- **Log rotation.** Before this fix, Docker's default settings kept the log file growing forever. `docker inspect qr-service --format '{{.HostConfig.LogConfig}}'` now shows `{json-file map[max-file:3 max-size:10m]}`.

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> (Invoke-RestMethod "http://localhost:9200/qr-service-logs-*/_count?q=log.level:*").count
37
PS C:\Users\IBM\Desktop\ESD-Assignment-01> (Invoke-RestMethod "http://localhost:9200/qr-service-logs-*/_count?q=log.level:*").count
37
PS C:\Users\IBM\Desktop\ESD-Assignment-01> (Invoke-RestMethod "http://localhost:9200/qr-service-logs-*/_count?q=log.level:*").count
39
PS C:\Users\IBM\Desktop\ESD-Assignment-01> (Invoke-RestMethod "http://localhost:9200/qr-service-logs-*/_count?q=log.level:*").count
40
PS C:\Users\IBM\Desktop\ESD-Assignment-01> docker compose up -d --force-recreate qr-service
```

Figure C6: Log count before and after recreating Filebeat (37 → 37), then +1 per new request.

How to search for a request or an error in Kibana

1. Open **Discover** and select the data view **QR Service Logs**.
2. **Set the time range first.** Kibana only searches inside it and displays local time. Most "No results" problems during this project came from the time range.

3. Type a KQL query and press Enter. The saved queries are exported, together with the data view, to `telemetry/kibana/saved-objects.ndjson` (5 objects):

Goal	KQL	Saved as
One request	<code>request.id:"<id from the X-Request-ID header>"</code>	Find request by ID
All errors	<code>log.level:"ERROR"</code>	Errors
Delayed requests (Part E)	<code>fault.delay_ms:500</code>	fault delay 500 ms
Fault on/off events (Part E)	<code>fault.every_n:*</code>	fault every n
Slow requests, any cause	<code>duration.ms >= 500</code>	not saved
Successful URL QR codes	<code>http.response.status_code:200 and qr.kind:"url"</code>	not saved

1. Click the arrow next to a document to see all of its fields (Table view) or the raw stored JSON (JSON view).

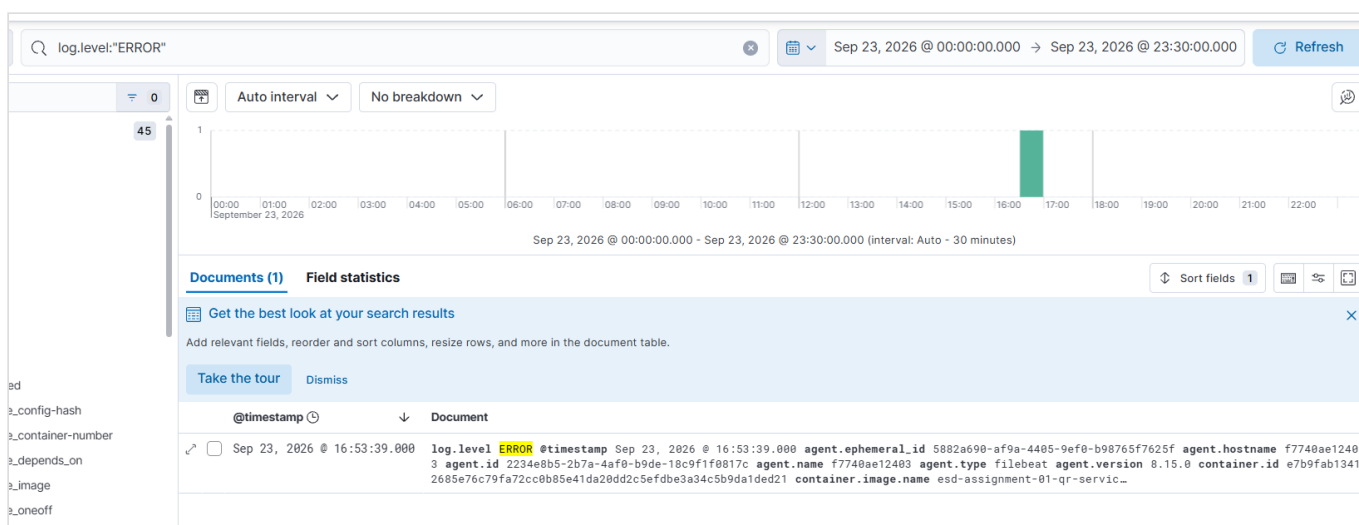


Figure C5: The saved search "Errors" (`log.level:"ERROR"`) finds the one failed request, a test request with no `text` sent at 16:53:39.

Part D: System design

1. Architecture diagram

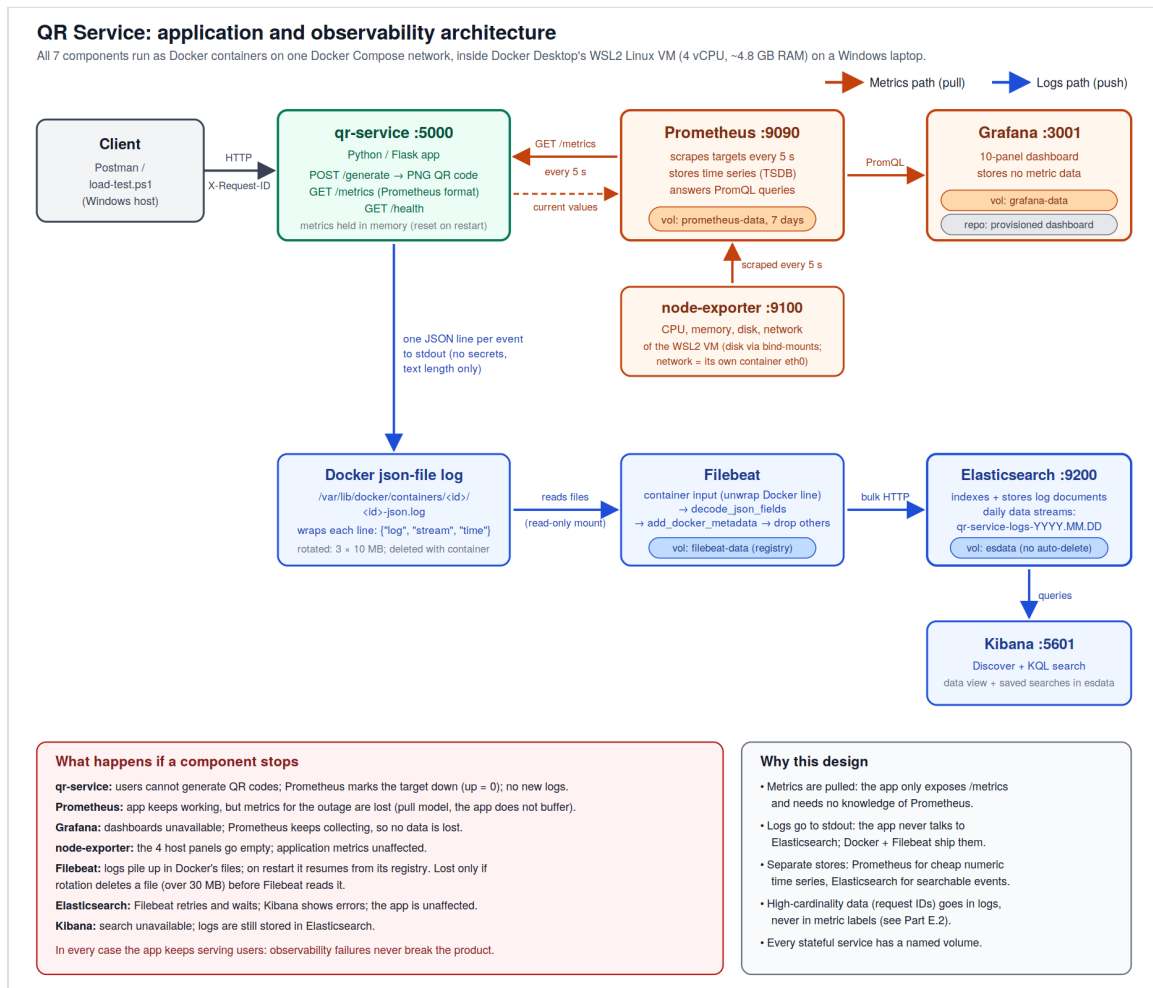


Figure D1: Architecture. Orange is the metrics path (pull), and blue is the logs path (push).

How the components communicate:

- **Client → qr-service:** HTTP on port 5000. The response carries X-Request-ID.
- **Prometheus → qr-service and node-exporter:** Prometheus pulls. Every 5 seconds it sends `GET /metrics` to each target and stores the numbers it receives. The app doesn't know Prometheus exists; it only answers the request.
- **Grafana → Prometheus:** Grafana sends PromQL queries whenever a dashboard is viewed or refreshed. Grafana stores no metric data itself.
- **qr-service → Docker → Filebeat:** the app prints to stdout. Docker writes those lines to a file, and Filebeat reads the file through a read-only mount.
- **Filebeat → Elasticsearch:** Filebeat pushes batches of documents over HTTP (the bulk API) on port 9200.
- **Kibana → Elasticsearch:** Kibana sends search queries when you use Discover.

Where data is stored: see the storage table in Part C for logs. For metrics, Prometheus keeps its time series in `prometheus-data` for 7 days. Grafana's own settings are in `grafana-data`, and its dashboard and datasource are loaded from the repo on every start. The app's metric values live only in its memory, which is why counters reset on restart. Every component that holds state has a named volume.

Why this design:

- **Metrics are pulled.** The app only needs to expose `/metrics`, with no Prometheus address, retries or buffering in the app. Prometheus also detects a dead target, because the scrape fails and `up = 0`.
- **Logs go to stdout.** The app never talks to Elasticsearch, so a logging outage cannot slow down or break the app. Docker and Filebeat handle shipping.
- **There are two separate stores, each suited to its data.** Prometheus is very efficient for numbers over time, like rates and percentiles. Elasticsearch is built for searching individual events by any field, like a request ID.
- **High-cardinality values such as request IDs go in logs, never in metric labels** (proven in Part E.2).
- **Grafana is provisioned from files.** Anyone who clones the repo gets the same dashboard.

What happens if a component stops:

Component stops	Effect
qr-service	Users cannot generate QR codes. Prometheus marks the target DOWN (<code>up = 0</code>), app panels show gaps, and no new logs appear.
Prometheus	The app keeps working, but metrics for the outage are lost , because nothing collects them and the app does not buffer. Grafana shows "No data".
Grafana	Dashboards are unavailable, but Prometheus keeps collecting, so no data is lost.
node-exporter	The four host panels go empty. Application metrics are unaffected.
Filebeat	Logs build up in Docker's files. When Filebeat returns, it resumes from its saved registry position, so nothing is lost unless rotation deletes a file (over 30 MB of new logs) first.
Elasticsearch	Filebeat keeps retrying and waits. Kibana shows errors. The app is unaffected.
Kibana	Log search is unavailable, but logs are still collected and stored.

In every case except the app itself, users are not affected. A failure in the monitoring stack never breaks the product.

Things I do not fully understand yet:

- How Prometheus's storage engine (TSDB) organises data internally, for example the in-memory "head block" and how blocks are compacted to disk.
- Exactly how much Filebeat buffers in memory when Elasticsearch is down, and when it starts to apply backpressure.
- Why Filebeat's Docker autodiscover did not detect containers on Docker Desktop/WSL2. I switched to a simpler approach instead of finding the root cause.
- The internals of Grafana's newer (v2) dashboard JSON format. I exported it and used it, but did not study its schema.

2. Following one metric and one log

Both traces follow the same request, sent from Postman at `17:56:52Z` (`22:56:52 local`) with body `{"text": "https://example.com"}`. It received request ID `db847888-78a4-4e80-97e0-ff07d833a356` (Figure D2). The app had just been restarted, so the counters had started at 0.

Key	Value
Server	Werkzeug/3.1.8 Python/3.12.14
Date	Wed, 23 Sep 2026 17:56:52 GMT
Content-Type	image/png
Content-Length	551
Cache-Control	no-cache
X-Request-Id	db847888-78a4-4e80-97e0-ff07d833a356
Connection	close

Figure D2: The traced request's ID.

Metric: `qr_generated_total{kind="url"}`

1. The code updates it. In `generate()` (lines 220–221, Figure D3), after the PNG is created:

```
kind = "url" if text.lower().startswith(("http://", "https://")) else "text"
qr_generated_total.labels(kind=kind).inc()
```

The text starts with `https://`, so `kind = "url"`, and the counter in the app's memory goes from 0 to 1.

2. The app exposes it. On `GET /metrics`, before the request (Figure D4a):

```
qr_generated_total{kind="url"} 0.0
```

After the request (Figure D4b):

```
qr_generated_total{kind="url"} 1.0
```

3. Prometheus collects and stores it. Within 5 seconds, Prometheus scrapes `/metrics` and stores the sample with a timestamp. While doing so, it **adds two labels** identifying where the data came from, `job` and `instance`. Querying `qr_generated_total{kind="url"}` in Prometheus returns (Figure D5):

```
qr_generated_total{instance="qr-service:5000", job="qr-service", kind="url"} 1
```

The sample is now in the TSDB on the `prometheus-data` volume and will be kept for 7 days.

4. Grafana queries and displays it. The "QR Codes Generated (Total)" panel runs `sum(qr_generated_total)` against Prometheus (Figure D6). `sum()` adds the `url` series (1) and the `text` series (0) and drops all labels, so the panel changes from 0 to **1** at the first step after the request (22:57:00 in Figure D6).

```
218     qr_generation_duration_summary.observe(duration)
219
220     kind = "url" if text.lower().startswith(("http://", "https://")) else "text"
221     qr_generated_total.labels(kind=kind).inc()
222
```

Figure D3: The code that increments the counter.

```
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1.048576e+06
# HELP qr_generated_total Total number of QR codes generated
# TYPE qr_generated_total counter
qr_generated_total{kind="url"} 0.0
qr_generated_total{kind="text"} 0.0
# HELP qr_generated_created Total number of QR codes generated
# TYPE qr_generated_created gauge
qr_generated_created{kind="url"} 1.7901848979581275e+09
qr_generated_created{kind="text"} 1.7901848979581788e+09
# HELP qr_generation_errors_total Total number of failed QR generation requests
# TYPE qr_generation_errors_total counter
qr_generation_errors_total{reason="missing_text"} 0.0
qr_generation_errors_total{reason="internal_error"} 0.0
# HELP qr_generation_errors_created Total number of failed QR generation requests
# TYPE qr_generation_errors_created gauge
```

```
process_open_fds 7.0
# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1.048576e+06
# HELP qr_generated_total Total number of QR codes generated
# TYPE qr_generated_total counter
qr_generated_total{kind="url"} 1.0
qr_generated_total{kind="text"} 0.0
# HELP qr_generated_created Total number of QR codes generated
# TYPE qr_generated_created gauge
qr_generated_created{kind="url"} 1.7901848979581275e+09
qr_generated_created{kind="text"} 1.7901848979581788e+09
# HELP qr_generation_errors_total Total number of failed QR generation requests
# TYPE qr_generation_errors_total counter
```

Figure D4: `/metrics` before (0.0) and after (1.0).

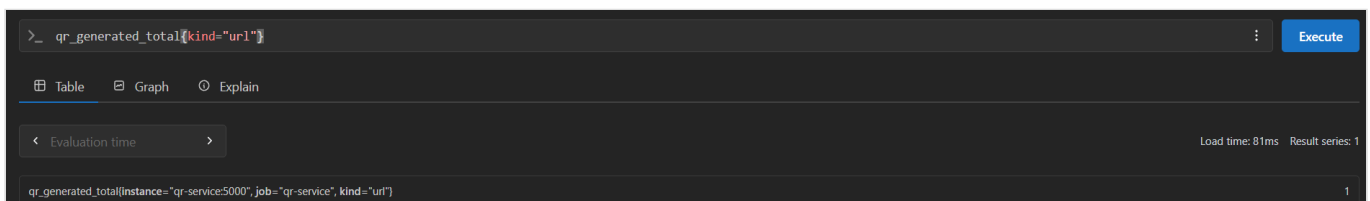


Figure D5: Prometheus has stored the value, with `job` and `instance` added.

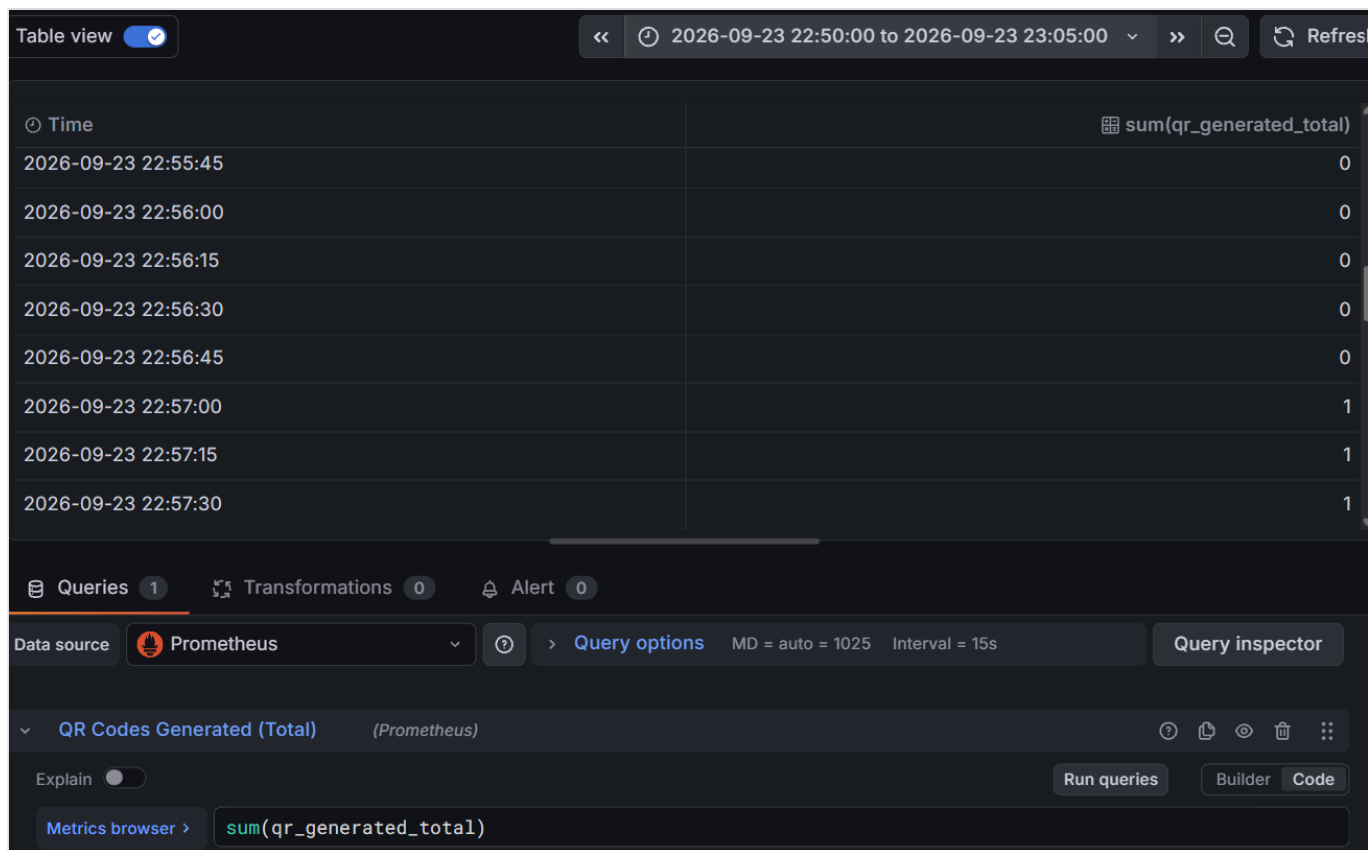


Figure D6: The Grafana panel in Table view with its query, `sum(qr_generated_total)`. The value is 0 up to 22:56:45 and 1 from 22:57:00; Grafana evaluates the query every 15 s, so the change appears at the first step after the scrape.

Log: the same request

1. **The code writes it.** `log_event()` prints one JSON line to stdout (Figure D7):

```
{ "@timestamp": "2026-09-23T17:56:52Z", "service.name": "qr-service", "log.level": "INFO",
  "message": "QR code generated successfully", "http.method": "POST", "http.path": "/generate",
  "http.response.status_code": 200, "duration.ms": 629.87, "request.id": "db847888-78a4-4e80-97e0-ff07d833a356", "qr.kind": "url", "text.length": 19 }
```

`duration.ms` is 629.87 because this was the first request after a restart, the cold-start effect from Part B.

2. **Docker saves it.** Docker's json-file driver **wraps our line** inside its own JSON. Our line becomes an escaped string in the `log` field, with a trailing newline, and Docker adds the stream name and its own nanosecond timestamp. Read directly from the file Filebeat reads (`docker exec filebeat sh -c 'grep 'db847888...' /var/lib/docker/containers/$id/$id-json.log'`, Figure D8):

```
{ "log": "{ \"@timestamp\": \"2026-09-23T17:56:52Z\", \"service.name\": \"qr-service\",
  \"log.level\": \"INFO\", \"message\": \"QR code generated successfully\", \"http.method\":
  \"POST\", \"http.path\": \" /generate\", \"http.response.status_code\": 200, \"duration.ms\":
  629.87, \"request.id\": \"db847888-78a4-4e80-97e0-ff07d833a356\", \"qr.kind\": \"url\",
  \"text.length\": 19 } \n", "stream": "stdout", "time": "2026-09-23T17:56:52.866635897Z" }
```

3. **Filebeat collects and parses it.** The container input unwraps `log` into `message`. `add_docker_metadata` adds the container details. `decode_json_fields` then turns our JSON string back into separate top-level fields, and overwrites `message` and `@timestamp` with our own values. One side

effect is visible: Docker's precise time 17:56:52.866635897Z is replaced by the app's second-precision 17:56:52.000Z. A millisecond timestamp in the app would avoid this.

4. Elasticsearch stores it. The document goes into the backing index `.ds-qr-service-logs-2026.09.23-2026.09.23-000001` of that day's data stream. Stored `_source`, shortened (copied from Kibana's JSON view of the document):

```
{
  "@timestamp": "2026-09-23T17:56:52.000Z",
  "service.name": "qr-service",
  "log.level": "INFO",
  "message": "QR code generated successfully",
  "http.method": "POST",
  "http.path": "/generate",
  "http.response.status_code": 200,
  "duration.ms": 629.87,
  "request.id": "db847888-78a4-4e80-97e0-ff07d833a356",
  "qr.kind": "url",
  "text.length": 19,
  "stream": "stdout",
  "container": { "name": "qr-service", "id": "9ef18d9d13ef...", "image": { "name": "esd-assignment-01-qr-service" } },
  "log": { "file": { "path": "/var/lib/docker/containers/9ef18d9d13ef.../9ef18d9d13ef...-json.log" }, "offset": 185 },
  "input": { "type": "container" },
  "agent": { "type": "filebeat", "version": "8.15.0" },
  "host": { "name": "f7740ae12403" }
}
```

`log.offset: 185` means this line started 185 bytes into the file. It was the second line in the new container's log. `host.name` is Filebeat's own container ID, not the app's.

5. Kibana finds it. The query `request.id:"db847888-78a4-4e80-97e0-ff07d833a356"` returns exactly this document (Figure D10). Kibana shows the time as 22:56:52 local.

Summary of how the log's format changes:

Stage	Format
App	One flat JSON object
Docker	Our JSON becomes an escaped string inside <code>{"log", "stream", "time"}</code>
Filebeat	Unwrapped, parsed back into fields, and enriched with container, agent and file fields; <code>@timestamp</code> replaced by the app's
Elasticsearch	Indexed document, where every field is searchable and numbers are stored as numbers

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> docker logs qr-service | Select-String "db847888-78a4-4e80-97e0-ff07d833a356"
{"@timestamp": "2026-09-23T17:56:52Z", "service.name": "qr-service", "log.level":
"INFO", "message": "QR code generated successfully", "http.method": "POST",
"http.path": "/generate", "http.response.status_code": 200, "duration.ms": 629.87,
"request.id": "db847888-78a4-4e80-97e0-ff07d833a356", "qr.kind": "url", "text.length":
19}
```

Figure D7: The log as written by the app.

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> $id = docker inspect -f "{{.Id}}" qr-service
PS C:\Users\IBM\Desktop\ESD-Assignment-01> docker exec filebeat sh -c "grep 'db847888-78a4-4e80-97e0-ff07d833a356' /var/lib/do
cker/containers/$id/$id-json.log"
{"log":{"@timestamp":"2026-09-23T17:56:52Z", "service.name":"qr-service", "log.level":"INFO", "message":"\nQ
R code generated successfully", "http.method":"POST", "http.path":"/generate", "http.response.status_code": 200,
"duration.ms": 629.87, "request.id":"db847888-78a4-4e80-97e0-ff07d833a356", "qr.kind":"url", "text.length": 19}
n", "stream":"stdout", "time":"2026-09-23T17:56:52.866635897Z"}
PS C:\Users\IBM\Desktop\ESD-Assignment-01>
```

Figure D8: The same line wrapped by Docker on disk.

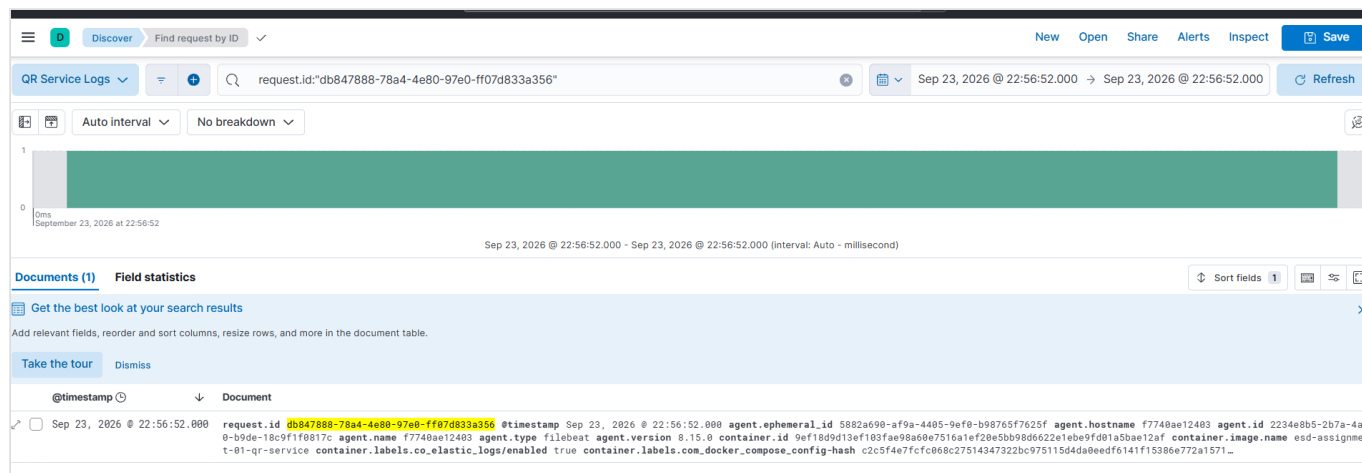


Figure D10: The saved search "Find request by ID" returns exactly this document.

Part E: Experiments

1. Reproducing a problem: slow requests

The fault: every fifth call to `/generate` sleeps for an extra 500 ms. The sleep happens inside the timed section, so it appears in the histogram, the summary and `duration.ms`, exactly like a real slowdown would.

It is switched on and off at runtime through `POST /admin/fault`, so the app is **never restarted** during the experiment. A restart would reset the counters and add a slow cold start, which would muddy the results. The gauge `qr_fault_injection_active` and a WARN/INFO log record exactly when the fault was on.

The repeatable test: `scripts/load-test.ps1` sends one request every 0.5 s for 6 minutes, alternating URL and text payloads, and prints a summary. Six minutes is longer than the 5-minute `rate()` window and covers about 70 Prometheus scrapes.

Step 1: Normal behaviour (baseline)

```
powershell -ExecutionPolicy Bypass -File scripts\load-test.ps1 -Minutes 6
```

Run from 21:33:14 to 21:39:15 (Figure E1). An earlier baseline run from 21:13 to 21:20 gave the same pattern (Figure E5), which shows the test is repeatable.

Step 2: Predictions (written before introducing the fault)

Signal	Prediction	Reason
p95 (Panel 3)	Rises from about 0.025 s to about 0.88 s	20% of requests take about 0.505 s and land in the 0.5–1 s bucket. Prometheus assumes they are spread evenly across that bucket: $0.5 + (0.95 - 0.80) \div 0.20 \times 0.5 \approx 0.875$ s. The estimate will be higher than the real value .
Average (Panel 4)	Rises by about 100 ms	$20\% \times 500$ ms
Throughput (Panel 5)	Falls slightly	The script waits for each response before sending the next
Error rate (Panel 6)	Stays at 0%	Requests are slow but still succeed
CPU (Panel 7)	Roughly unchanged	<code>sleep</code> waits without using the CPU
Logs	About 20% of success logs have <code>duration.ms ≥ 500</code> and a <code>fault.delay_ms</code> field; no ERROR logs	
Timing	Charts take up to 5 minutes to fully reflect a change	5-minute <code>rate()</code> window

Step 3: Introducing the problem

```
Invoke-RestMethod -Method Post http://localhost:5000/admin/fault -ContentType "application/json" -Body '{"every_n":5,"delay_ms":500}' powershell -ExecutionPolicy Bypass -File scripts\load-test.ps1 -Minutes 6
```

Run from 21:52:52 to 21:58:54 (Figures E2 and E3).

Results

Stage	Window (local)	Requests	Client avg	Client p95	Client ≥ 400 ms	Server p95 (Panel 3)	Server avg (Panel 4)
Baseline	21:33:14–21:39:15	600	90 ms	227 ms	16 (3%)	≈ 0.025 s	≈ 0.010–0.015 s
Fault on	21:52:52–21:58:54	533	168 ms	589 ms	108 (20%)	≈ 0.875 s	≈ 0.11 s
Recovery	22:07:21–22:13:23	587	105 ms	314 ms	24 (4%)	≈ 0.025 s	≈ 0.012–0.015 s

The Prometheus queries behind the server columns are:

```
histogram_quantile(0.95, rate(qr_generation_duration_seconds_bucket[5m]))
rate(qr_generation_duration_summary_seconds_sum[5m]) /
rate(qr_generation_duration_summary_seconds_count[5m])
```

What actually changed, compared with the predictions:

- **p95 went to about 0.875 s, exactly as calculated**, even though the delayed requests really took only about 0.505 s (for example, one log shows `duration.ms: 506.54` with `fault.delay_ms: 500`). This confirms that histogram percentiles are estimates limited by the bucket boundaries. A bucket at 0.6 s would give a much more accurate p95 for this fault.
- **The average rose from about 0.01 s to about 0.11 s**, matching the predicted +100 ms.
- **Exactly 20% of requests were slow** (108 of 533), one in every five.

- **Throughput fell from 600 to 533 requests** in 6 minutes, and Panel 5 peaked at about 44 per minute per type instead of 50.
- **Error rate stayed at 0%**, with 0 failures in every stage.
- **CPU showed no increase, as predicted** (Figure E6). It was about 20% during the fault stage, compared with 25–35% during the baselines, slightly lower because fewer requests were processed. Host metrics alone would not reveal this problem.
- **The in-progress gauge picked up the fault**, which I had not predicted. `qr_in_progress` (Figure E5, bottom right) touched 1 several times between 21:55 and 21:57. At baseline a request lasts about 10 ms, so a scrape almost never catches one in progress. Delayed requests last 0.5 s, so a scrape sometimes lands inside one.
- **Logs:** Kibana `fault.delay_ms:500` returns 110 documents: 108 delayed requests plus 2 "switched ON" events, which also contain that field. The bar chart shows them only between about 21:52 and 21:59 (Figure E7). `fault.every_n:*` shows the switch events (Figure E8). The ON/OFF pair at about 21:40 was an aborted first attempt, when the fault was switched off again before any load was sent.
- **Timing differed slightly from the prediction.** p95 jumped straight to 0.875 s instead of rising gradually. There was a pause of more than 5 minutes between stages, so the 5-minute window never mixed baseline and fault requests. Running the stages back to back would show a gradual transition.

Client-side versus server-side numbers: even at baseline, the script measured p95 = 227 ms, while the app's own histogram shows about 25 ms. The difference is time spent outside the app: PowerShell's `Invoke-WebRequest` plus the network hop from Windows into Docker's VM. The app's metrics cannot see that part, and it also explains the 16 "slow" client requests at baseline when nothing was wrong. The server-side metrics isolate the app, and they show the fault clearly.

Step 4: Cause and effect on users

The cause is the injected `time.sleep(0.5)` on every fifth request, which stands in for a real problem like a slow dependency, lock contention or a slow disk.

The effect on users: 80% of users notice nothing, but one user in five waits an extra half second for their QR code. The average (0.11 s) still looks acceptable and hides this, while p95 (about 0.9 s) exposes it immediately. That is why latency should be monitored with percentiles, not averages. The error rate stays at 0%, so an alert based only on errors would never fire. A p95 latency alert, for example "p95 > 0.5 s for 5 minutes", would catch this problem. Host CPU gives no hint either. Only application metrics and logs reveal it, and the logs give the exact request IDs affected.

Step 5: Removing the problem and checking recovery

```
Invoke-RestMethod -Method Post http://localhost:5000/admin/fault -ContentType "application/json"
-Body '{"every_n":0,"delay_ms":0}'
powershell -ExecutionPolicy Bypass -File scripts\load-test.ps1 -Minutes 6
```

Run from 22:07:21 to 22:13:23 (Figure E4b). Requests went back to 587 (baseline 600). The ≥ 400 ms share fell from 20% back to 4%, close to the 3% client-side noise at baseline, and there were no new `fault.delay_ms` logs after the OFF event. On the server side, p95 returned to about 0.025 s and the average to about 0.012–0.015 s between 22:08 and 22:13, the same as the baselines (Figure E5).

The p95 and average panels show a short spike at about 22:16–22:18. It happened after the recovery load had ended at 22:13:23, so it is not part of the fault. As the 5-minute window moved

past the end of the test, it contained fewer and fewer requests, only the last ones sent. At the same time, CPU on the VM rose to about 55% because of other activity on the laptop, so those final requests were a little slower. With only a handful of requests left in the window, those few dominated the result. This is the same low-traffic effect seen after cold starts in Part B: percentiles computed from very few requests are unreliable. The fault was fully undone with one command, and no restart was needed.

```
==== Summary ====
Window:          21:33:14 -> 21:39:15
Requests:        600 (failures: 0)
Average:         90 ms (client-side, includes network)
p95:             227 ms
Slow (>=400ms): 16 (3%)
PS C:\Users\IBM\Desktop\ESD-Assignment-01>
```

Figure E1: Baseline summary.

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> Invoke-RestMethod -Method Post http://localhost:5000/admin/fault -ContentType "application/json" -Body '{"every_n":5,"delay_ms":500}'

delay_ms every_n
-----
500      5
```

Figure E2: Fault switched on.

Figure E3: Fault-stage summary (script output):

```
==== Summary ====
Window:          21:52:52 -> 21:58:54
Requests:        533 (failures: 0)
Average:         168 ms (client-side, includes network)
p95:             589 ms
Slow (>=400ms): 108 (20%)
```

```
PS C:\Users\IBM\Desktop\ESD-Assignment-01> Invoke-RestMethod -Method Post http://localhost:5000/admin/fault -ContentType "application/json" -Body '{"every_n":0,"delay_ms":0}'

delay_ms every_n
-----
0        0
```

Figure E4a: Fault switched off.

```

===== Summary =====
Window:      22:07:21 -> 22:13:23
Requests:    587 (failures: 0)
Average:     105 ms (client-side, includes network)
p95:        314 ms
Slow (>=400ms): 24 (4%)
PS C:\Users\IBM\Desktop\ESD-Assignment-01> |

```

Figure E4b: Recovery summary.



Figure E5: Application panels from 21:10 to 22:20: two baselines (21:13, 21:33), the fault stage (21:52–21:59) and recovery (22:07–22:13).



Figure E6: Host panels over the same range. CPU does not rise during the fault stage.

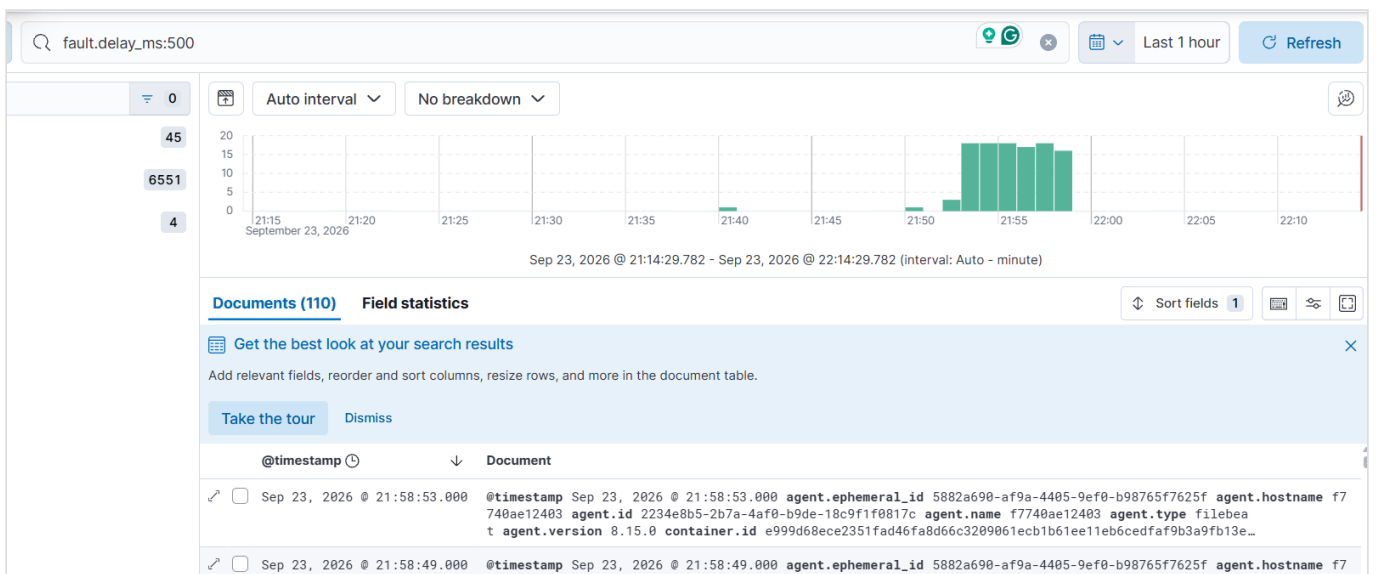


Figure E7: Kibana, delayed requests only during the fault stage, with one expanded document.

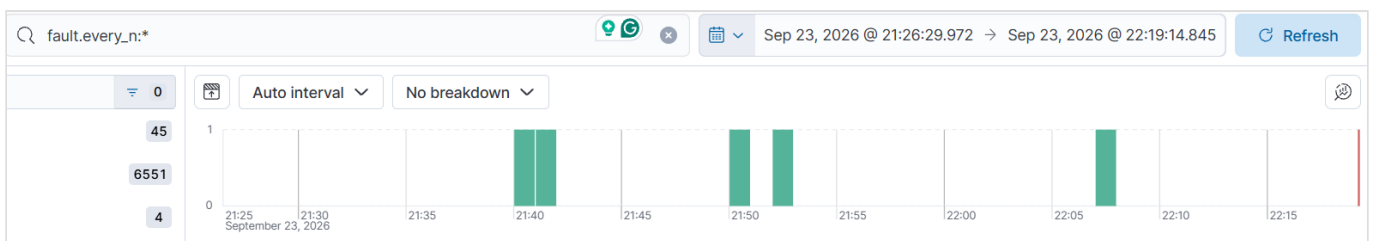


Figure E8: Kibana, fault ON/OFF events.

2. Cardinality explosion

Setup: a separate test counter, `demo_requests_total`, on the `/demo` endpoint. When the environment variable `DEMO_REQUEST_ID_LABEL=true` is set, it gets a `request_id` label, so every request creates a new

time series. Otherwise it has no labels. The code refuses new IDs after 100, so the experiment cannot grow out of control.

Step 1: Add a `request_id` label and stop at 100 IDs.

```
$env:DEMO_REQUEST_ID_LABEL="true"
docker compose up -d --build qr-service
1..100 | ForEach-Object { Invoke-WebRequest -Method Post http://localhost:5000/demo -
UseBasicParsing | Out-Null }
```

Step 2: Show the series growth (after more than 2 scrapes, Figures E9 and E9b):

```
count(demo_requests_total)           → 100
scrape_samples_scraped{job="qr-service"} → 242
```

Step 3: Remove the label, restart and repeat.

```
$env:DEMO_REQUEST_ID_LABEL="false"
docker compose up -d qr-service
1..100 | ForEach-Object { Invoke-WebRequest -Method Post http://localhost:5000/demo -
UseBasicParsing | Out-Null }
```

After another scrape (Figure E10):

```
count(demo_requests_total)           → 1
scrape_samples_scraped{job="qr-service"} → 44
count(count_over_time(demo_requests_total[30m])) → 101
```

	Series	Samples per scrape
With <code>request_id</code> label	100	242
Without label	1	44

What the numbers show:

- 100 requests with 100 unique IDs created **100 separate series**, each with a value of 1 that will never be updated again. Without the label, one series counts all 100 requests.
- Samples per scrape fell by **198, not 100**. The Python client exports a `demo_requests_created` timestamp series for every label value too, so each ID really cost 2 samples: 42 other samples + $100 \times 2 = 242$, and $42 + 2 = 44$. The real cost is double what the series count suggests.
- **Removing the label did not delete the old data.** `count_over_time(...[30m])` still finds 101 series: the 100 old ones plus the new one. The old series are simply no longer scraped. They stay on disk until the 7-day retention removes them.

Step 4: The cost at larger scale, and why IDs belong in logs.

At a modest 100 requests per second, a request-ID label would create 8.6 million new series per day, doubled to 17 million samples by the `_created` series. Prometheus keeps an index entry for every series and holds recently active series in memory, so memory use would climb until Prometheus slowed down or crashed. Every query touching the metric would have to scan millions of series, dashboards would time out, and storage would fill with series that each hold a single, never-updated value. Prometheus's own guidance is to avoid labels with unbounded values such as user IDs, email addresses or request IDs.

Request IDs belong in logs. A log already stores one document per request, so adding the ID costs nothing extra, and Elasticsearch is designed to find one document among millions by an exact field value (`request.id:"..."` in Part C). Metrics answer "how many, how fast, how often" across all requests. Logs answer "what happened to this one request". The `kind` label in this project is safe for the opposite reason: it only ever has two values.

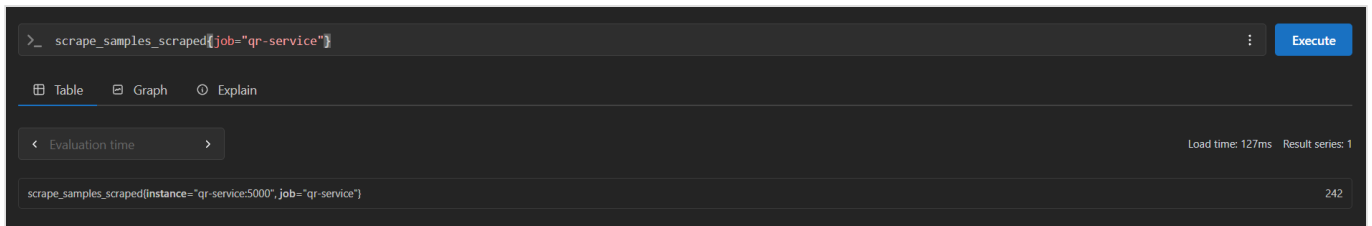


Figure E9: 242 samples per scrape with the `request_id` label.



Figure E9b: `count(demo_requests_total)` over time: 100 with the label, then 1 after it was removed. The Prometheus UI shows UTC, so 17:36 is 22:36 local.

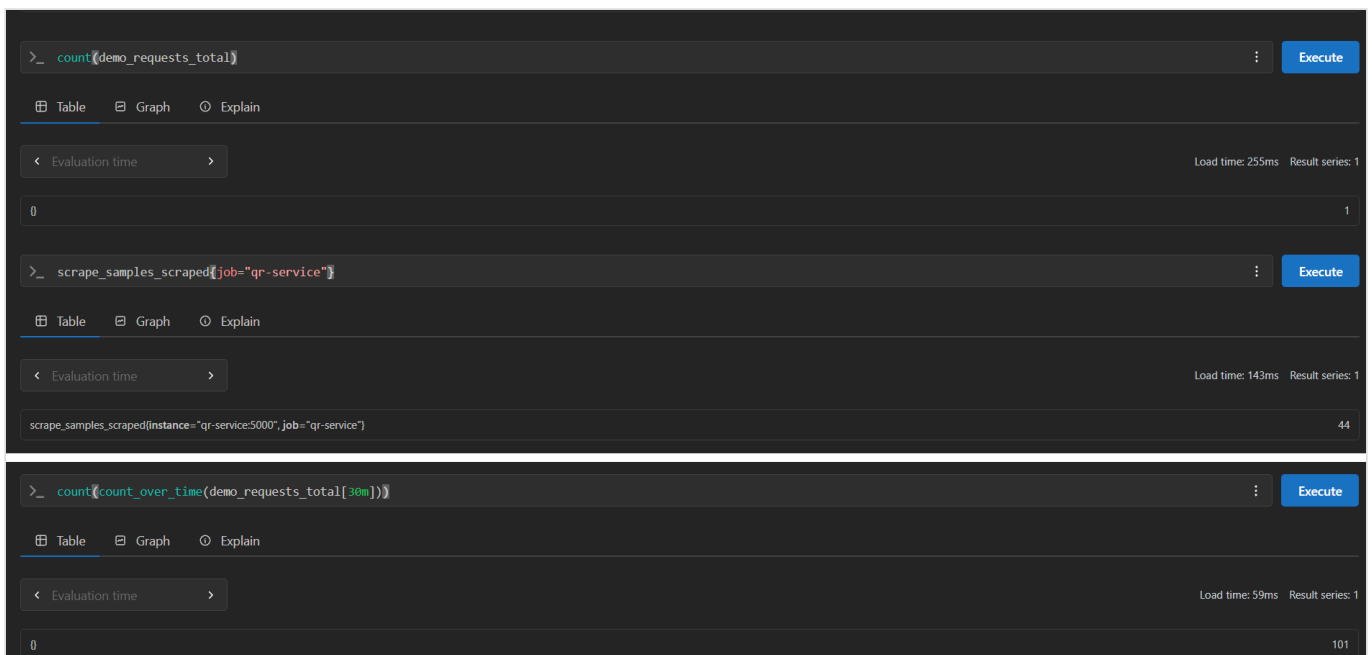


Figure E10: 1 series after removing the label, 44 samples per scrape, and 101 series still in history.